

PW EARNERS
DSA in C++

Daily Practice Problems (DPP) — 5

Topic: Functions and Pointers in C++

For Q1 to Q15: Predict the exact output of each C++ program block. Note that 'return 0;' has been omitted intentionally, which is valid in modern C++. Read the concepts carefully!

Q1.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 10;
5     {
6         int x = 20;
7         cout << x << " ";
8     }
9     cout << x;
10 }
```

Output:

.....

Q2.

```
1 #include<iostream>
2 using namespace std;
3 int val = 50;
4 int main(){
5     int val = 15;
6     cout << val << " " << ::val;
7 }
```

Output:

.....

Q3.

```
1 #include<iostream>
2 using namespace std;
3 void printDetails(int a, int b = 10, int c = 20) {
4     cout << a << " " << b << " " << c << endl;
5 }
6 int main(){
7     printDetails(5);
8     printDetails(5, 15);
9 }
```

Output:

.....

Q4.

```
1 #include<iostream>
2 using namespace std;
3 void modify(int x) {
4     x = x + 10;
5 }
6 int main(){
7     int a = 5;
8     modify(a);
9     cout << a;
10 }
```

Output:

.....

Q5.

```
1 #include<iostream>
2 using namespace std;
3 void modifyRef(int &x) {
4     x = x + 10;
5 }
6 int main(){
7     int a = 5;
8     modifyRef(a);
9     cout << a;
10 }
```

Output:

.....

Q6.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int score = 40;
5     int* ptr = &score;
6     *ptr = 99;
7     cout << score;
8 }
```

Output:

.....

Q7.

```
1 #include<iostream>
2 using namespace std;
3 void display(int a) { cout << "Int: " << a << endl; }
```

```
4 void display(double a) { cout << "Double: " << a << endl; }
5 int main(){
6     display(7);
7     display(7.5);
8 }
```

Output:

.....

Q8.

```
1 #include<iostream>
2 #include<cmath>
3 using namespace std;
4 int main(){
5     int x = 36, y = 15;
6     cout << max((int)sqrt(x), y);
7 }
```

Output:

.....

Q9.

```
1 #include<iostream>
2 using namespace std;
3 int square(int n) { return n * n; }
4 int addFive(int n) { return square(n) + 5; }
5 int main(){
6     cout << addFive(4);
7 }
```

Output:

.....

Q10.

```
1 #include<iostream>
2 using namespace std;
3 void update(int* p) {
4     *p = *p * 2;
5 }
6 int main(){
7     int x = 12;
8     update(&x);
9     cout << x;
10 }
```

What will be the output of the above code?

- A. 12 B. 24 C. Address of x D. Error

Q11.

```
1 #include<iostream>
```

```

2 using namespace std;
3 int main(){
4     int a = 8;
5     int* p1 = &a;
6     int** p2 = &p1;
7     cout << **p2;
8 }

```

What does **p2 print?

- A. Address of a B. Address of p1 C. 8 D. Garbage Value

Q12.

```

1 #include<iostream>
2 using namespace std;
3 int num = 100;
4 void compute(int num) {
5     num = num + ::num;
6     cout << num << " ";
7 }
8 int main(){
9     compute(20);
10    cout << num;
11 }

```

Output:

.....

Q13.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 10, y = 20;
5     int *p = &x;
6     p = &y;
7     *p = 50;
8     cout << x << " " << y;
9 }

```

Output:

.....

Q14.

```

1 #include<iostream>
2 using namespace std;
3 int globalVar = 10;
4 void mysteryFunction(int* ptr, int &ref) {
5     *ptr = ref + globalVar;
6     ref = *ptr - 5;
7     globalVar++;
8 }
9 int main(){

```

```

10     int a = 20, b = 30;
11     mysteryFunction(&a, b);
12     cout << a << " " << b << " " << globalVar;
13 }

```

Output:

.....

Q15.

```

1  #include<iostream>
2  using namespace std;
3  int& trickyUpdate(int &x, int y) {
4      x = x + y;
5      return x;
6  }
7  int main(){
8      int val = 5;
9      trickyUpdate(val, 3) = 25;
10     cout << val;
11 }

```

Output:

.....

Q16. State **True** or **False** for each statement below regarding C++ Functions and Pointers.

- (i) Default arguments in a C++ function must always be provided from right to left.
- (ii) The statement `int* p = &x;` stores the value of variable `x` directly inside `p`.
- (iii) Function overloading allows you to have two functions with the exact same name and parameter list, as long as their return types are different.
- (iv) Using the scope resolution operator `::x` allows you to access a global variable `x` even if a local variable `x` is shadowing it.
- (v) A double pointer `int** p` is used to store the address of a regular integer variable directly.
- (vi) The ampersand operator (`&`) is used to get the memory address of a variable.
- (vii) The inbuilt math function `sqrt(25)` returns an integer data type by default in C++.

For Q17 to Q30: Write clean, modular C++ code. Use proper functions wherever instructed. Handle input/output gracefully.

Q17. Write a program containing a function `findMaximum(int a, int b, int c)` that uses the inbuilt `max()` function to return the largest of three numbers. Call this function in `main()`.

Sample — Input: 12 45 9 $\Rightarrow$ Output: 45

Q18. Write a function `printEven(int start, int end)` that prints all even numbers between `start` and `end` (inclusive). Call it from `main()`.

Sample — Input: 10 15 $\Rightarrow$ Output: 10 12 14

Q19. Write a C++ program to swap two numbers using a function `swapNumbers(int* a, int* b)`. You must use pointers (pass by address) to perform the swap.

Sample — Input: 5 9 $\Rightarrow$ Output: 9 5

Q20. Write a program demonstrating **Function Overloading**. Create two functions named `calculateArea`. One should take a single `double` parameter (radius) and return the area of a circle. The other should take two `double` parameters (length, width) and return the area of a rectangle.

Sample — Input: Circle radius 3 $\Rightarrow$ Output: 28.26
Input: Rectangle 4 5 $\Rightarrow$ Output: 20

Q21. Write a program to calculate the hypotenuse of a right-angled triangle given its base and perpendicular. Write a function that accepts these two values and uses the inbuilt `sqrt()` library function to return the hypotenuse.

Sample — Input: 3 4 $\Rightarrow$ Output: 5

Q22. Write a function `sumFirstAndLast(int n)` that extracts the first and the last digit of a given positive integer and returns their sum.

Sample — Input: 4529 $\Rightarrow$ Output: 13 (Since $4 + 9 = 13$)

Q23. Write a function `updateValues(int *a, int *b)`. The function should update the value of `a` to the sum of `a` and `b`, and update the value of `b` to the absolute difference of their original values. (*Hint: Store actual values in temp variables first!*)

Sample — Input: `a = 4, b = 5` $\Rightarrow$ Output after function call: `a = 9, b = 1`

Q24. Write a program to print **Pascal's Triangle** up to `N` rows. You **MUST** create a helper function `factorial(int x)` and a combination function `nCr(int n, int r)` to calculate the values in the triangle.

Sample — Input: 4 $\Rightarrow$ Output:

```
1
1 1
1 2 1
1 3 3 1
```

Q25. Write a function `reverseNumber(int n)` that takes an integer, reverses its digits mathematically, and uses the `return` keyword to give back the reversed integer to `main()`.

Sample — Input: 1234 $\Rightarrow$ Output: 4321

Q26. Write a boolean function `isPrime(int n)` that checks if a number is prime. Then, in `main()`, use this function in a loop to print all prime numbers between 1 and a user-provided integer `N`.

Sample — Input: 10 $\Rightarrow$ Output: 2 3 5 7

Q27. Create a program that demonstrates **Double Pointers**. Write a function `changeTarget(int** ptr, int* newAddress)` that makes a pointer in `main` point to a completely different variable's address.

Sample context — `int a = 10, b = 20; int *p = &a;` Pass `p` to the function so that it points to `b` instead.

Q28. Create a mini banking application demonstrating **Global Variables**. Declare a global variable `double balance = 0.0;`. Write two functions `deposit(double amount)` and `withdraw(double amount)` that modify this global state. Print the final balance from `main()`.

Sample — `deposit(500), withdraw(200)` $\Rightarrow$ Output: Balance: 300

Q29. Write a function `sortTwo(int &a, int &b)` using **Pass by Reference**. The function should check the values, and if `a > b`, it should swap them. This ensures the variables in `main` are always ordered smallest to largest after the call.

Sample — Input: 15 6 $\Rightarrow$ Output variables print as: 6 15

Q30. Write a program containing nested functions. Create a function `findPermutation(int n, int r)` to calculate $nPr = \frac{n!}{(n-r)!}$. This function must internally call another function `getFactorial(int x)`. Take `n` and `r` as inputs from the user.

Sample — Input: `n = 5, r = 2` $\Rightarrow$ Output: 20

All the best! Keep Coding ♡